WHERE

Tuple Sieve & Three-Valued Logic (3VL) Predicate Filter

SQL FOUNDRY // ARCHITECTURE MANUAL

Phase 2 (Evaluated immediately after FROM / JOIN)

Complexity: O(N) full scan without index; O(log N) + O(K) index seek with B-Tree.

1. ANSI Syntax & Grammar Specification

```
WHERE boolean_expression
    [ AND | OR [NOT] boolean_expression ... ]
```

Filters rows emitted by the FROM clause before any aggregation, grouping, or projection occurs. Operates under ANSI Three-Valued Logic (TRUE, FALSE, UNKNOWN). A row is admitted into subsequent pipeline stages ONLY if its predicate evaluates strictly to TRUE; rows evaluating to FALSE or UNKNOWN (NULL comparisons) are discarded.

2. Database Engine Internals & Storage Footprint

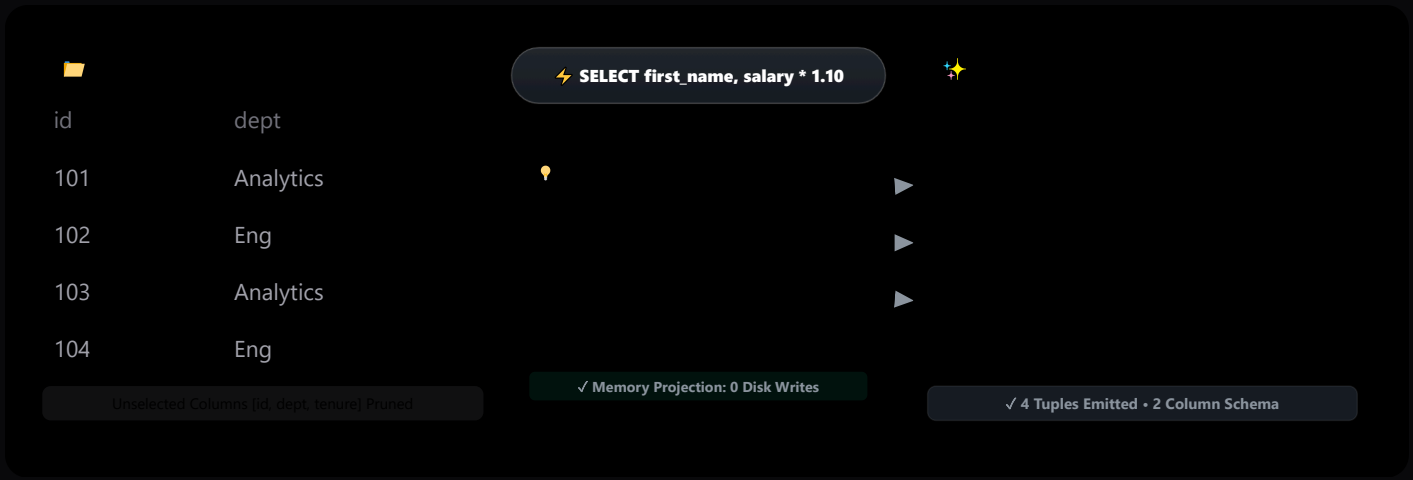
Physical Operator: Streaming / Row-by-Row Sieve

Memory Footprint: O(1) memory register per candidate tuple.

Disk Spill Risk: None. Sieve evaluates in-flight in CPU L1/L2 cache registers.

Index SARGability: Critically dependent on SARGability (Search Argument Able). Functions applied to indexed columns (e.g. WHERE YEAR(date) = 2026) destroy B-Tree index seeks and degrade to table scans.

3. Relational Architecture Diagram (Tuple Lifecycle)



Overview | 1. Scan Table | 2. Relational Engine | 3. Emit Result

WHERE // IMPLEMENTATION PATTERNS

MODULE: PREDICATE LOGIC & SIFTING

PAGE 2 OF 4

4. Canonical Query Code Progression

Pattern A: SARGable Range and Exact Match Predicates

```
-- Clean SARGable predicates utilizing index seeks
SELECT emp_id, first_name, salary, dept
FROM enterprise_employees
WHERE dept = 'Engineering'
  AND salary >= 100000;
```

Pattern B: Date Boundary & Multi-Condition Logic

```
-- SARGable date filtering avoiding function wrapping on indexed column
SELECT order_id, customer_id, order_date, total_amount
FROM customer_orders
WHERE order_date >= '2026-01-01'
  AND order_date < '2026-02-01'
  AND status IN ('COMPLETED', 'SHIPPED')
  AND canceled_at IS NULL;
```

Pattern C: Correlated Subquery & Anti-Join Sieve

```
-- High-performance subquery predicate filtering active high-value customers
SELECT c.customer_id, c.account_name, c.tier
FROM enterprise_accounts c
WHERE c.tier = 'PLATINUM'
  AND EXISTS (
      SELECT 1
      FROM trade_executions t
      WHERE t.account_id = c.customer_id
        AND t.trade_timestamp >= CURRENT_TIMESTAMP - INTERVAL '30' DAY
        AND t.notional_usd >= 1000000
  );
```

5. Production Schemas & Input/Output Relational Tables

Input: 4 Candidate Rows

first_name	department	salary	status
David	Engineering	$125,000	Active
Ashley	Analytics	$115,000	Active
Carlos	Engineering	$72,000	Active
Sara	Engineering	NULL	On Leave

Emitted Output Table

first_name	department	salary
David	Engineering	$125,000

WHERE // INDUSTRY APPLICATION

COMPANY APPLICATION • INTERVIEW RIGOR

6. Production Enterprise Case Study

Company / Team: JPMorgan Chase Financial Crimes & Anti-Money Laundering (AML)

Operational Scenario: Global compliance monitors SWIFT wire transfers exceeding $250,000 destined for non-FATF compliant jurisdictions, screening out domestic clearing house records in real-time.

```
SELECT
    wire_reference_id,
    originator_account,
    beneficiary_account,
    amount_usd,
    beneficiary_country_code
FROM swift_wire_transfers
WHERE amount_usd >= 250000.00
  AND beneficiary_country_code NOT IN ('US', 'GB', 'DE', 'FR', 'JP', 'CA')
  AND compliance_review_status = 'PENDING_SCREEN'
  AND ofac_match_score IS NOT NULL;
```

Business & Infrastructure Impact: Filters 99.4% of benign wire traffic in Phase 2 before expensive forensic fuzzy string matching, preserving sub-millisecond payment authorization latencies.

7. FAANG & Wall Street Technical Interview Defense

[JPMorgan / Goldman Sachs] Question 1:

What is Three-Valued Logic (3VL), and what happens when you run "WHERE salary <> 50000" on a table where some salary values are NULL?

Model Answer: ANSI SQL evaluates Boolean expressions to TRUE, FALSE, or UNKNOWN. When comparing a column to a scalar and that column is NULL (NULL <> 50000), the result is UNKNOWN, not TRUE. Because WHERE only retains tuples evaluating strictly to TRUE, all rows with NULL salaries are silently discarded. To include them, you must write: WHERE salary <> 50000 OR salary IS NULL.

[Apple / Meta] Question 2:

Explain SARGability. Why is "WHERE created_at >= '2026-01-01'" vastly faster than "WHERE YEAR(created_at) = 2026"?

Model Answer: SARGable stands for Search Argument Able. When a column is wrapped in a function like YEAR(created_at), the database engine cannot predict the output without executing the function across every single row, invalidating the B-Tree index and forcing a full table scan. The range predicate allows the engine to seek directly to the exact B-Tree leaf node in O(log N) time.

WHERE // DEFENSE AUDIT & MCQS

3VL TRAPS • CERTIFICATION AUDIT

PAGE 4 OF 4

8. High-Yield Pitfalls & Corporate Production Bugs

Critical **The NOT IN (NULL) Trap:** If a subquery inside NOT IN returns even a single NULL value (e.g., WHERE id NOT IN (SELECT supervisor_id FROM employees)), the entire IN expression evaluates to UNKNOWN for every candidate row, returning 0 rows across the whole query! Always use NOT EXISTS instead.

High **Non-SARGable Wildcards:** Writing "WHERE email LIKE '%@gmail.com'" forces a full table scan because the leading wildcard prevents the B-Tree index from sorting by prefix.

9. Certification & Conceptual Mastery Examination (5 MCQs)

1. What does the predicate "WHERE NULL = NULL" evaluate to in ANSI SQL?

A) TRUE

B) FALSE

C) UNKNOWN

D) Syntax Error

Correct Answer: C • *In relational algebra, NULL represents an unknown or missing value. Unknown cannot be confirmed equal to another unknown, so NULL = NULL evaluates to UNKNOWN, which WHERE discards.*

2. Why does "SELECT COUNT(*) FROM t WHERE x NOT IN (1, 2, NULL)" return 0 regardless of table data?

A) The query parser encounters a fatal error

B) "x NOT IN (1, 2, NULL)" expands to "x <> 1 AND x <> 2 AND x <> NULL". Any comparison with NULL yields UNKNOWN, making the overall conjunction UNKNOWN for all rows

C) NULL is treated as zero

D) NOT IN cannot take integer arguments

Correct Answer: B • *In 3VL, "AND UNKNOWN" can never resolve to TRUE. Since every row produces UNKNOWN, 0 rows pass the WHERE filter.*

3. Which of the following WHERE conditions is fully SARGable for an index on column "signup_date"?

A) WHERE DATE(signup_date) = '2026-05-01'

B) WHERE signup_date + INTERVAL '1' DAY > '2026-05-01'

C) WHERE signup_date >= '2026-05-01' AND signup_date < '2026-05-02'

D) WHERE TO_CHAR(signup_date, 'YYYY-MM-DD') = '2026-05-01'

Correct Answer: C • *Option C leaves the column untransformed on the left side of the comparison operators, allowing direct B-Tree index seeks.*

4. Can you filter the result of an aggregate function (e.g. SUM(salary) > 500000) inside a WHERE clause?

A) Yes, if placed at the end of the WHERE clause

B) No, because WHERE executes at Phase 2, before GROUP BY and aggregation occur at Phase 3/4

C) Yes, but only in SQLite and PostgreSQL

D) Only if combined with an index

Correct Answer: B • *WHERE filters individual base tuples before groups are formed. Post-aggregation filtering must be performed in the HAVING clause (Phase 5).*

5. In short-circuit evaluation of "WHERE A AND B", what does the optimizer do if predicate A evaluates to FALSE?

A) Always evaluates B anyway

B) Skips evaluation of predicate B immediately, saving CPU cycles

C) Raises a warning

D) Re-evaluates A with opposite polarity

Correct Answer: B • *Modern query execution engines short-circuit Boolean evaluation. In FALSE AND [anything], the result is guaranteed to be FALSE regardless of B, so B is bypassed.*